Machine Exercise 1

Discrete-Time Signals, Systems, and Random Processes

Signal generation, sampling, BIBO stability via difference equations, impulse response and pole-zero analysis, fourier series synthesis, stationarity and ergodicity in random processes

Machine Exercise

August 24, 2026

Contents

1.1 Signal generation **4**

1.2 Discrete-Time System Representations **4**

1.3 Examples of DT Systems **5**

1.4 Signal synthesis using Fourier analysis **6**

1.5 Frequency analysis of signals **7**

1.6 Random Process **8**

Reflection and Academic Integrity Declaration **10**

INSTRUCTIONS. This is an open-notes, open-books exercise to be done individually. You are to submit a lab report detailing the process and analysis of your chosen approach, and an executable Jupyter notebook or a MATLAB Live script `.mlx` file. Make sure to write your name and student number in the first part of your submission so that it is easily identifiable to you. Include figure, plots, diagrams, tables, etc. as necessary in your discussion to clarify your answers and analysis of the results.

From your `.mlx` or `.ipynb`, generate the PDF output with comments, plots, answers to the questions and discussion. Submit as a single zipped file the `.mlx` or `.ipynb` and the the PDF output. If there are other external files needed (*.csv, *.wav, etc) include everything in the zipped file and upload to the UVLe submission bin before the deadline. Anything submitted after the deadline can still be graded but with a 0.6 multiplier penalty to the final score. The submitted zipped file must be easily identifiable to you, use the suggested file name `<Surname>_AI321_ME01.zip` when submitting any machine exercise.

Some parts depend on your student number

Every student works with a seed and a few parameters derived deterministically from their own official student number, see the code snippet below. This means some specifications of the exercise are unique for each student. For this machine exercise, add one verification step that depends on your own student number, in the pole-zero and BIBO-stability results: using `params['offset_small']`, perturb one denominator coefficient a_k by a small amount (e.g. `a1_perturbed = a1 * (1 + offset_small/100)`) and repeat your impulse-response, pole-zero, and BIBO-stability analysis on this perturbed system too. Report the effect of the perturbed coefficient and compare with the original.

```python
PYTHON

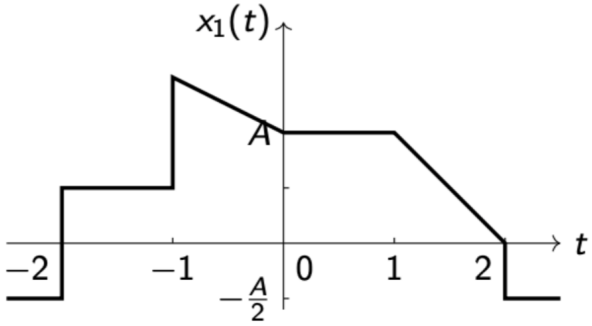
import hashlib
import numpy as np

def get_my_params(student_number):
    """AI 321 / EE 298 -- personalized exercise parameters.
    Deterministic, unique per official student number. Run once at the
    top of your notebook/script; use the returned fields wherever an
    exercise below tells you to."""
    digest = hashlib.sha256(str(student_number).strip().encode()).hexdigest()
    seed = int(digest[:8], 16)
    return {
        "student_number": str(student_number).strip(),
        "seed": seed,
        "rng": np.random.default_rng(seed),
        "offset_small": seed % 30,        # e.g. a day/index offset, 0-29
        "offset_large": seed % 3000,      # e.g. a row offset into a long series
        "subset_size": 30 + (seed % 21),  # e.g. 30-50, for "sample N series" tasks
        "choice_of_8": seed % 8,          # e.g. selecting 1 of 8 provided variants
    }
params = get_my_params("2021-XXXXX")  # <-- replace with YOUR official student number
```

A MATLAB equivalent (`get_my_params.m`) is provided alongside this exercise for students working in MATLAB.

1.1 Signal generation

Given the periodic analog signal $x_1(t)$ as shown below, where t is in seconds, and use the value $A = 2$.

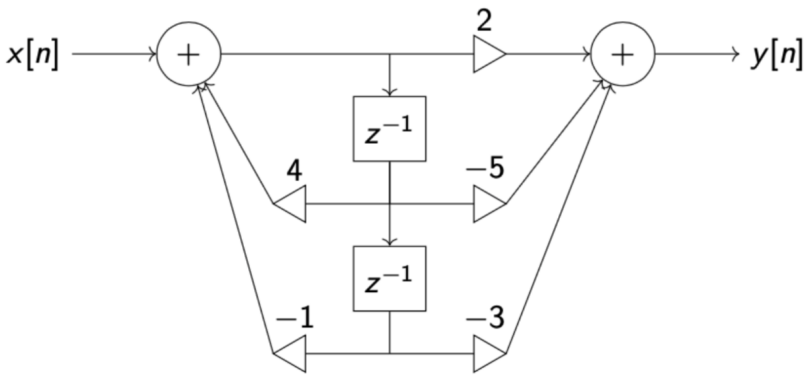


a. Generate the stem plot of the discrete-time equivalent signal $x(n)$ of $x_1(t)$ using a sampling frequency of 10 samples per second. You are to estimate the value of the signal at $t = -0.5$, given the relative scaled of the axes. You are free to use any available functions in the Scipy Library or MATLAB toolbox.

b. Generate the stem plot of the odd $(x_o(n))$ and even $(x_e(n))$ components of the signal $x(n)$.

1.2 Discrete-Time System Representations

Given the Direct Form II representation of a discrete time system below, we want to determine and prove using MATLAB or Python whether the system is BIBO stable or not. The different representations of a discrete-time system can help determine if the system is stable or not.



a. **Difference equation**. From the Direct Form II realization of the system, determine the difference equation that represents the system. What are the coefficients of the moving average (MA) part (b_k coefficients) and the autoregressive (AR) part (a_k coefficients).

b. Using the recursive nature of the difference equation, implement the difference equation in MATLAB or Python (using for loops) with simple test inputs to determine if the output is bounded. You can look at the examples given in #1.3 below on how to work out the recursion. Is the system BIBO stable via recursion? Why or why not?

c. **Impulse response**. Determine the impulse response of the system, using the direct

method of computation from the difference equation.

i. Using the computed impulse response, and using an impulse as an input, calculate the first 15 samples of the output, $y(n)$.

ii. From the plot of $y(n)$, is the system BIBO stable? Why or why not?

MATLAB can approximate the impulse response from the coefficients of the system function, using the b_k and a_k coefficients, using the `impz()` function. Then with a simple impulse signal as input, a 1 followed by many zeros `xn = [1 0 0 0 0 …]`, you can determine the impulse response of the system plot the output $y_1(n)$. Compare the resulting output with the one computed using the direct method, are they the same? Is the system computed using `impz()` BIBO stable? Why or why not?

d. **Pole-Zero plot**. From the coefficients of the system function, b_k and a_k, you can also plot the poles and zeros of the system using `pzmap()`. From the location of the poles and zeros of the system is the system BIBO stable? Why or why not?

1.3 Examples of DT Systems

Discrete time systems can be represented using block diagrams and difference equations. Shown below is an example of a discrete time ARMA system:

$$y[n] = x[n] + 5x[n-1] + 2y[n-1]$$

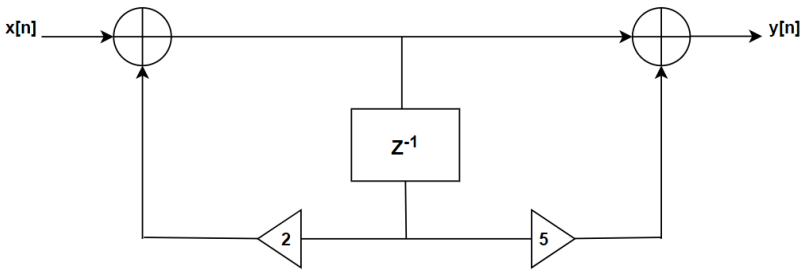


Figure 1.1: Direct Form II representation of the system

In MATLAB, we usually use the following vectors to represent a discrete time system:

$$a_0 y[n] + a_1 y[n-1] + \cdots + a_N y[n-N] = b_0 x[n] + b_1 x[n-1] + \cdots + b_M x[n-M]$$

$$\text{num}/b = [\, b_0 \; b_1 \; b_2 \; \cdots \; b_M \,]$$

$$\text{den}/a = [\, a_0 \; a_1 \; a_2 \; \cdots \; a_N \,]$$

In the given example, we have:

`num / b = [1 5]` `den / a = [1 -2]`

The following is a MATLAB code example on how to implement a system and plot the system response. You can use the example codes given to answer the exercise #1.2.

Using recursion

We can simulate the recursion using the for loop function. For the code below, the simulation runs on the entire domain of n:

$$y[n] = x[n] + 5x[n-1] + 2y[n-1]$$

```MATLAB
x = randn(1,5); %random input signal x
y = zeros(1,length(x)); %initialize output signal y
for n = 1:length(x)  %5 time indices
  if n<2
      y(n) = x(n);
  else
      y(n) = x(n) + 5*x(n-1) + 2*y(n-1);
  end
end

figure(1);

subplot 211
stem(1:5,x); title('input signal');

subplot 212
stem(1:5,y); title('output signal');
```

Using the impulse response and DT convolution

We can also simulate the response using the `impz()` function and `conv()`:

```MATLAB
b = [1 5]; a = [1 -2];
h = impz(b,a); %impulse response
y = conv(h,x);

figure(2);

subplot 211
stem(1:5,x(1:5)); title('input signal');

subplot 212
stem(1:5,y(1:5)); title('output signal');
```

1.4 Signal synthesis using Fourier analysis

Given the rectangular pulse signal shown below and using the given example MATLAB code on discrete fourier series synthesis, determine the first ten (10) complex fourier coefficients of the signal, and plot the magnitude and angle on the same figure window. Use the following parameters: $F_s = 200$ Hz. Plot the original signal and the reconstructed signal using 10, 15 and 25 fourier coefficients. Discuss the quality of the reconstruction as the number of coefficients increase. The synthesis and analysis equations of the Fourier series for periodic signals is given below.

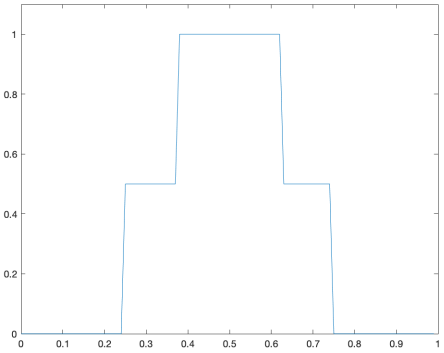


Figure 1.2: Rectangular Pulse Signal to be synthesized using Fourier series

Table 1.1: Frequency Analysis of Discrete-Time Periodic Signals

Synthesis	$x(n) = \sum_{k=0}^{N-1} c_k e^{j2\pi kn/N}$	(16)
Analysis	$c_k = \frac{1}{N} \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N}$	(17)

1.5 Frequency analysis of signals

Generate a signal that is a sum of the four sinusoids given below:

$x_1(t) = 1.5\cos(120\pi t)$	$x_3(t) = 1.3\sin(1400\pi t + 30^\circ)$
$x_2(t) = 1.0\sin(700\pi t + 150^\circ)$	$x_4(t) = 2.1\cos(1024\pi t - 45^\circ)$

Use a sampling rate $F_s = 2500$ Hz, and a signal duration of 0.25s. Plot the signal $x_s(t)$ as the sum of the four sinusoids. Plot the one-sided discrete Fourier transform, with `Nfft = 512`, of the signal and verify that frequencies are consistent. Calculate the frequency resolution of the resulting discrete-frequency spectrum and based on your analysis of the peaks of the signal spectrum, what are the resulting frequencies of the given signal. Explain the discrepancy of the frequencies in spectrum and known signal frequencies.

Increase the signal duration to 0.5s, and again plot the one-sided DFT of the signal, with `Nfft = 1024`. Determine the new frequency resolution of the signal spectrum and based on the analysis of the peaks, were you able to improve the estimate of the signal frequencies?

The computation of the discrete-time Fourier transform (DFT) and its inverse if given by the following equations.

The DFT

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N}, \qquad k = 0, 1, 2, \ldots, N-1$$

The IDFT

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j2\pi kn/N}, \qquad n = 0, 1, 2, \ldots, N-1$$

This is also implemented in MATLAB as the `fft()` function.

1.6 Random Process

In this exercise we will only deal with discrete-random processes, those processes that have the form $x(0), x(1), x(2), \ldots, x(t)$, where $t = 0, 1, 2, \ldots, N$. In other textbooks the time variables is also denoted by n: $x[1], x[2], x[3], \ldots, x[n]$, where $n = 0, 1, 2, \ldots, N$. In this exercise both notations mean the same.

A random process in discrete time is a sequence of random variables. Thus, a process $\{x[n]\}$ has actually two dimensions: the time variable n takes values from $\ldots, -1, 0, 1, \ldots$, whereas the realization is chosen from the continuous "event space", according to the specified distribution. The simplest random process is the White Gaussian Noise (WGN), which is a sequence of uncorrelated random variables with a Gaussian distribution.

To illustrate the two-dimensional nature of a stochastic process, we can think of an $N \times K$ matrix X.

$$X = \begin{bmatrix} x_1[1] & \cdots & x_K[1] \\ \vdots & \ddots & \vdots \\ x_1[N] & \cdots & x_K[N] \end{bmatrix}$$

Thus, the nth row of X contains K different realizations of a sample $x[n]$, whereas the column k is one realization of the whole sequence $\{x[1], \ldots, x[N]\}$, indexed by the "realization number" k. Thus, we can form the *Ensemble Average* $\mu_x[n]$ of a sample $x[n]$ as

$$\mu_x[n] = \lim_{K \to \infty} \frac{1}{K} \sum_{k=1}^{K} x_k[n]$$

and the *Time Average* for the kth realization is given by

$$\bar{x}_k = \lim_{N \to \infty} \frac{1}{N} \sum_{n=1}^{N} x_k[n]$$

An important concept for stochastic process is ergodicity, which means that ensemble and time average gives the same result. For this to be possible we must clearly have $\mu_x[n] = \mu_x$, independent of n, and $\bar{x}_k = \bar{x}$, independent of k. Thus, ergodicity is related to (wide-sense) stationarity.

The statistics of a stationary random process do not change with time. The statistics of an ergodic process can be computed from the statistics of a single sample function.

A. Let $x(t) = X_t$ where X_t is a sequence of independent, identically distributed, Gaussian random variables with mean 0 and variance 1. This process is an example of a Gaussian process. Generate and plot ten (10) sample functions $x(t, \rho_0), x(t, \rho_1), \ldots$ up to $x(t, \rho_{10})$ of this random process, for $t = 0$ to 999. You can generate these two sample functions in Matlab using the statement.

```
x = randn(1000,10)
```

where each column contains a sample function. You can plot the ten (10) sample functions with the statement:

```
figure; plot(0:999,x)
```

B. Plot the ensemble average $\hat{\mu}_x(t)$ as a function of t by averaging along the rows in X. Use `mean(X')`. Next, plot the time average as a function of realization (`mean(X)`). Show that this process is stationary.

C. Show that this process is ergodic. Generate 1000 sample functions of this random process, for $t = 0$ to 999. Compute an estimate for the process mean:

$$\eta(t) = E[x(t)], \qquad \text{for } t = 0, \dots, 999.$$

Compute an estimate for the mean per sample

$$\eta(\rho_i) = \frac{1}{1000} \sum_{t=0}^{999} x(t; \rho_i), \qquad \text{for } i = 0, \dots, 999.$$

Compare the two means.

The statistics for an ergodic process are more easily estimated than for a non-ergodic process, since in the former we do not need to generate many sample functions to estimate our statistics. For this reason, many problem solutions and algorithms assume ergodicity.

D. Consider the process $x(t) = 2\sin(2\pi \cdot 0.002\, t) + N_t$, for $t = 0, 1, \dots, 999$, where N_t is a sequence of independent, identically-distributed Gaussian random variables with mean 0 and variance 1. Plot five (5) sample functions of this process. Show that the process is non-stationary and non-ergodic. Use MATLAB (or python) to compute an estimate of the process mean and plot the process mean $\eta(t)$ as well as the mean per sample $\eta(\rho_i),\ i = 0, 1, \dots, 999$. Note that ergodicity implies stationarity but not the other way around.

Reflection and Academic Integrity Declaration

This section is required and graded on the depth and specificity of your answers, not on whether or how much you used AI. This responses in this section can be considered when grading the exercise as a whole.

AI 321's policy on AI tools — transparency, not prohibition

You are free to use AI tools (Claude, ChatGPT, Copilot, etc) to help you learn, debug, and understand concepts in this course. What you can not do is submit AI-generated code, derivations, or written answers as your own work without disclosing them and demonstrating you understand them. Doing so is academic dishonesty, equivalent to plagiarism, whether or not the code runs correctly.

1. Approach Summary

In 3-5 sentences, describe what you did and why, in your own words, as if explaining it to a classmate who has not seen this exercise.

[*Response*]

2. Key Insight

What is one result or observation from this exercise that surprised you, or didn't match your initial intuition?

[*student response*]

3. Most Difficult Part

What part of this exercise took the longest to get right, and how did you resolve it?

[*student response*]

4. Disclosure on the use of AI systems

List every part of this exercise where you used an AI tool, one row per use. Write "None" in the first row if you did not use AI at all. The last column is the part that can be considered in grading this exercise. Be clear on your statements, on how AI actually helped you if it did. They do make mistakes, you can also list here the instances where the AI failed and you were able to recognize its failure.

Section / task	**Tool used**	**What you asked it for**	**Explain the result in your own words, as if teaching it**
e.g., Problem 2, debugging	*e.g., ChatGPT*	*e.g., fix a shape mismatch in my autocorrelation matrix*	[*student response*]

5. Self-Assessed Understanding

For each objective or major problem in this exercise, rate your confidence (Confident / Mostly confident / Still shaky) and add one sentence why.

Objective / problem	Confidence	Why (one sentence)

6. Academic Honor Pledge

By submitting this exercise, I hereby affirm upon my honor the following:

1. The analysis, results, codes, answers to questions, discussions and conclusions in this machine exercise reflect my own understanding and work.
2. I used my own official student number whenever it is required, and the results generated were computed from my own personalized parameters/data, and not from anyone else.
3. I have disclosed fully and accurately any use of AI tools, LLMs, AI agents or similar systems in Section 4 above, and I can explain, in my own words, every part of what I submitted if asked.
4. I understand that submitting AI-generated work as my own can be considered as plagiarism and is an academic dishonesty, and that I abide with the University's policy on Academic Integrity as written in the Student's Code of Conduct.

Signed (name): _____ **Student number:** _____

Date: _____